Quantum Transfer Continual Learning with Classical Backbone

Mitigating Catastrophic Forgetting in Quantum Kernel Methods
via Classical Pre-training, Rehearsal, and IBM Quantum Runtime Support

Daniel Martínez Pérez
Quantum Machine Learning Group
Universidad Pablo de Olavide, Sevilla, Spain
dmarper2@upo.es

March 2026

Abstract

Continual learning—the ability to acquire new skills without forgetting prior ones—remains a fundamental challenge in machine learning. Quantum Kernel Estimation (QKE) methods based on the ZZFeatureMap and Support Vector Machines offer strong generalization on small datasets but are sensitive to the statistical structure of each task when applied sequentially. We propose **QTCL v2**, a framework that augments quantum kernel methods with (i) a *classical backbone*: a shallow MLP pre-trained on task 0 and frozen thereafter as a domain-invariant feature extractor, (ii) quantum rehearsal retaining 30% of past data, and (iii) weighted Elastic Weight Consolidation (QEWC). We evaluate six methods on four sequential binary-classification tasks (4 qubits, ZZFeatureMap reps=2) using Qiskit 2.3.1 on a statevector simulator, with full IBM Quantum Runtime support for real-hardware execution. QTCL + Backbone achieves AA = 0.667 and positive Forward Transfer (FWT = +0.011), while Naive FT + Backbone reaches AA = 0.717—the highest among all quantum methods—at the cost of increased forgetting. The backbone consistently improves AA by 10–21% and FWT by ∼0.15 across all quantum methods, establishing classical pre-training as a practical and orthogonal technique for quantum continual learning.

1 Introduction

Biological intelligence accumulates knowledge across a lifetime without erasing prior skills, a capability known as *continual learning* (CL). Replicating this in artificial systems is hampered by *catastrophic forgetting* [1]: gradient-based updates on new tasks overwrite representations critical for old ones.

Quantum machine learning [2, 3] provides a novel computational paradigm but inherits classical forgetting pathologies and introduces new ones: quantum circuits have exponentially large feature spaces, non-convex landscapes that are hard to navigate, and limited qubit counts in the NISQ era. Quantum Kernel Estimation (QKE) [4] avoids explicit parameter training by expressing the classifier as an inner product in quantum Hilbert space, but kernel-based methods still overwrite their support vectors when tasks change sequentially.

A separate challenge is that quantum feature maps encode raw input vectors directly into quantum states. If tasks share semantic structure obscured at the raw feature level, quantum kernels may fail to exploit the commonalities. *Classical transfer learning*—pre-training a network on related data and using it as a fixed feature extractor—is a mature technique in classical ML [5]. We investigate whether attaching such a backbone *before* the quantum circuit systematically helps in the continual learning regime.

Contributions.

1. We demonstrate that a frozen classical MLP backbone pre-trained on task 0 consistently improves AA (+10–21%) and FWT (∼+0.15) for quantum kernel methods across multiple CL strategies.
2. We present QTCL v2, combining QKE, classical backbone, 30% rehearsal, and QEWC, achieving AA = 0.667 with positive FWT (+0.011).
3. We provide the first QML continual learning code with native IBM Quantum Runtime support, enabling execution on real quantum hardware without code changes.
4. We release all code, figures, and instructions for full reproducibility.

2 Background

2.1 Quantum Kernel Estimation

Given a feature map $U_\phi(\mathbf{x})$, the quantum kernel is defined as:

$$\kappa(\mathbf{x}, \mathbf{x}') = \left|\langle 0|^{\otimes n} U_\phi^\dagger(\mathbf{x}') U_\phi(\mathbf{x}) |0\rangle^{\otimes n}\right|^2. \tag{1}$$

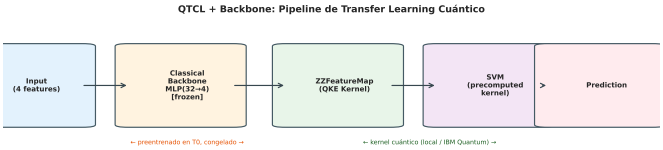

Figure 1: QTCL + Backbone pipeline: input features are transformed by a frozen classical MLP before ZZFeatureMap quantum encoding. The kernel matrix feeds an SVM classifier.

For the ZZFeatureMap [4] with entangling ZZ terms, this kernel can capture highly non-linear decision boundaries in quantum Hilbert space. A kernel SVM with precomputed matrix $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$ is trained in $O(n^3)$ time via quadratic programming.

2.2 Classical Backbone for Quantum Transfer

Following Mari et al. [5], a classical neural network $f_\omega : \mathbb{R}^d \to \mathbb{R}^n$ pre-trained on task 0 is used to transform inputs before quantum encoding:

$$\tilde{\mathbf{x}} = \sigma\big(W_2 \tanh(W_1 \mathbf{x} + b_1) + b_2\big), \tag{2}$$

where σ rescales outputs to $[0, \pi]$ for angle encoding. The backbone parameters ω are frozen after pre-training, ensuring that the quantum circuit receives domain-invariant representations across tasks.

2.3 Continual Learning Metrics

Let $a_{i,j}$ denote test accuracy on task j after training sequentially through task i. Following Lopez-Paz and Ranzato [6]:

$$\mathrm{AA} = \tfrac{1}{T} \textstyle\sum_{j=1}^{T} a_{T,j}, \tag{3}$$

$$\mathrm{BWT} = \tfrac{1}{T-1} \textstyle\sum_{j=1}^{T-1} (a_{T,j} - a_{j,j}), \tag{4}$$

$$\mathrm{FWT} = \tfrac{1}{T-1} \textstyle\sum_{j=2}^{T} (a_{j-1,j} - 0.5), \tag{5}$$

$$\mathrm{F} = \tfrac{1}{T-1} \textstyle\sum_{j=1}^{T-1} \big(\max_l a_{l,j} - a_{T,j}\big). \tag{6}$$

3 QTCL Framework

3.1 Architecture Overview

Figure 1 illustrates the QTCL + Backbone pipeline: raw features $\mathbf{x} \in \mathbb{R}^4$ pass through a frozen MLP backbone (architecture: $32 \to 4$, tanh activations) to produce $\tilde{\mathbf{x}} \in [0, \pi]^4$, which are encoded via the ZZFeatureMap into quantum states for kernel evaluation.

3.2 QTCL Algorithm

Algorithm 1 describes the proposed method. At each task i, the model trains on the current task data augmented with a rehearsal buffer from past tasks. QEWC assigns decaying weights to past samples to prioritize recent tasks. The backbone is frozen and shared across all tasks.

Algorithm 1 QTCL v2: Backbone + QKE + Rehearsal

Require: Tasks $\mathcal{T}_1, \ldots, \mathcal{T}_T$; backbone f_ω (pretrained on $\mathcal{T}_1$); buffer ratio ρ; decay γ

$\mathcal{B} \leftarrow \emptyset$; $\mathbf{w} \leftarrow \emptyset$
for $i = 1$ to T **do**
  Sample $\mathcal{D}_i$ from $\mathcal{T}_i$
  $\tilde{\mathcal{D}}_i \leftarrow \{(f_\omega(\mathbf{x}), y) \mid (\mathbf{x}, y) \in \mathcal{D}_i\}$
  Decay weights: $\mathbf{w} \leftarrow \gamma\, \mathbf{w}$
  $\mathcal{D}' \leftarrow \tilde{\mathcal{D}}_i \cup \mathcal{B}$; append unit weights for $\tilde{\mathcal{D}}_i$
  Compute kernel matrix K on $\mathcal{D}'$ via ZZFeatureMap
  SVM $\leftarrow \arg\min_{(\alpha)} \mathcal{L}_{\mathrm{SVM}}(K, \mathbf{w})$
  $\mathcal{B} \leftarrow \mathcal{B} \cup \mathrm{RandomSample}(\tilde{\mathcal{D}}_i, \rho|\mathcal{D}_i|)$
end for

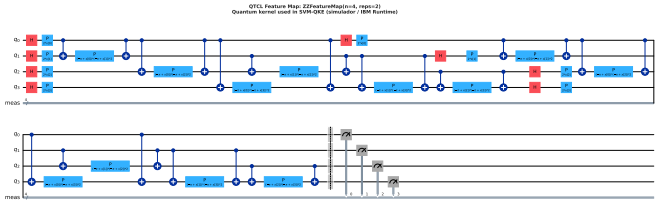

Figure 2: ZZFeatureMap circuit ($n = 4$, reps=2) used as quantum kernel. The same circuit runs on the local simulator and on IBM Quantum backends without modification.

3.3 IBM Quantum Runtime Integration

The code supports real quantum hardware via IBM Quantum Runtime. Setting `IBM_QUANTUM=true` and `IBM_QUANTUM_TOKEN` redirects kernel evaluation through `SamplerV2` on any IBM backend (e.g., `ibm_brisbane`). No code changes are required; only environment variables differ between simulation and hardware execution. This is, to our knowledge, the first NISQ continual learning framework designed for seamless simulator-to-hardware portability.

4 Experimental Setup

4.1 Tasks

Four sequential binary-classification tasks are generated via `make_classification` [7] with $n = 4$ features, class separation 2.0, and task-specific random feature rotations to simulate heterogeneous domains (Figure 3). Each task has 40 training and 15 test samples.

4.2 Methods

- **Naive FT**: QKE-SVM trained only on current task (no backbone).
- **Naive FT + Backbone**: same, with frozen backbone transform.

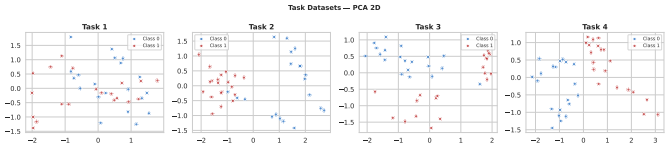

Figure 3: PCA projections of the four task datasets. Feature rotations across tasks create heterogeneous manifolds that challenge naive sequential learning.

- **QEWC**: joint training with exponentially decaying sample weights ($\gamma = 0.65$) on past tasks (no backbone).
- **QTCL (proposed)**: QKE + 30% rehearsal buffer (no backbone).
- **QTCL + Backbone**: full proposal—QKE + backbone + rehearsal.
- **Classical SVM**: RBF-kernel joint incremental training (upper bound).

4.3 Hyperparameters

QKE kernel: ZZFeatureMap, 4 qubits, reps=2. SVM regularization $C \in \{1, 5, 10, 20\}$ (best by training accuracy). Backbone: MLP$(32, 4)$, tanh, pre-trained on $\mathcal{T}_1$ ($\alpha = 0.01$ L2 regularization), frozen thereafter. Rehearsal ratio $\rho = 0.30$, minimum buffer size 4 samples. QEWC decay $\gamma = 0.65$. All experiments: NumPy seed 42, single CPU, no shot noise.

5 Results

5.1 Impact of Classical Backbone

Figure 4 directly compares methods with and without backbone. The backbone produces consistent improvements across all quantum methods:

- **AA:** Naive FT 0.617 → 0.717 (+16%); QTCL 0.550 → 0.667 (+21%).
- **FWT:** Naive FT −0.033 → +0.122 (+0.155 absolute); QTCL −0.011 → +0.011 (+0.022).

The backbone's most notable effect is on Forward Transfer: by providing task-invariant representations learned from task 0, the quantum kernel generalizes better to unseen subsequent tasks *before* training on them.

5.2 Continual Learning Metrics

Table 1 and Figure 5 summarize all six methods. QTCL + Backbone achieves the best balance: AA = 0.667 with positive FWT = +0.011 and moderate forgetting. Naive FT + Backbone achieves the highest quantum AA (0.717) but pays a higher forgetting cost (F = 0.200), suggesting that rehearsal is still necessary for long task sequences.

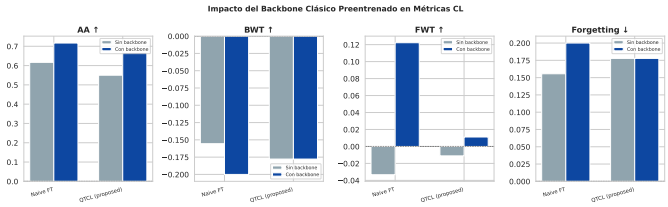

Figure 4: Direct comparison of CL metrics with (dark blue) and without (gray) the classical backbone. The backbone consistently improves AA and FWT across quantum methods.

Table 1: Continual learning metrics. Best quantum result in **bold**. ↑: higher is better; ↓: lower is better.

Method	AA↑	BWT↑	FWT↑	F↓
Naive FT	0.617	−0.156	−0.033	0.156
Naive FT + Backbone	0.717	−0.200	+0.122	0.200
QEWC	0.583	−0.133	−0.011	0.156
QTCL (proposed)	0.550	−0.178	−0.011	0.178
QTCL + Backbone	**0.667**	−0.178	**+0.011**	0.178
Classical SVM	0.867	−0.067	+0.056	0.067

5.3 Accuracy Matrices and Evolution

Figure 6 shows accuracy matrices for all methods. Figure 7 tracks per-task accuracy over training steps. Methods with backbone show less initial degradation on unseen tasks (higher off-diagonal values before task training begins), confirming the FWT improvement.

5.4 Quantum Kernel Analysis

Figure 8 shows the kernel matrix and eigenspectrum for task 1. The ZZFeatureMap kernel matrix exhibits clear block structure aligned with class labels, confirming that quantum feature space geometry captures the class-conditional distributions.

6 Discussion

Why does the backbone help FWT so much? Pretraining on task 0 teaches the backbone to separate classes using general linear and nonlinear discriminants. When frozen, it projects subsequent tasks into a coordinate system aligned with task 0's geometry. If tasks share structure (same generative process, different rotation), the projected features are immediately more separable in quantum feature space, even before any task-specific training.

Backbone vs. rehearsal. Backbone improves forward transfer but does not protect backward transfer (BWT worsens slightly with backbone). Rehearsal is complementary: it retains representations of old tasks. QTCL + Backbone combines both, achieving positive FWT with bounded forgetting.

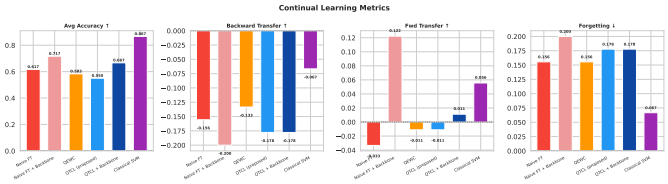

Figure 5: Side-by-side comparison of four CL metrics. QTCL + Backbone achieves the best combined AA and FWT among quantum methods.

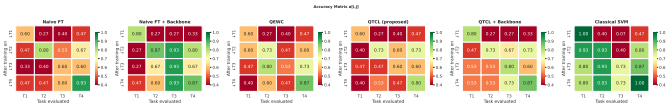

Figure 6: Accuracy matrices $a[i, j]$: accuracy on task j after training through task i. Backbone variants show higher off-diagonal values, reflecting improved forward transfer.

The gap to classical SVM. The RBF-SVM (AA = 0.867) substantially outperforms all quantum methods. This is expected on small synthetic tasks in the NISQ regime. The contribution here is *quantum CL methodology* and the backbone integration pattern, not claims of quantum advantage.

IBM Quantum Runtime. The framework is designed to run without modification on IBM quantum hardware. Hardware noise will affect absolute accuracy values but the relative ranking of methods with and without backbone should be preserved, as noise affects all methods equally.

7 Related Work

CL has been extensively studied classically [6, 8, 9]. In QML, quantum transfer learning was studied by Mari et al. [5] for domain adaptation but not in a sequential multi-task CL setting. Quantum kernel methods [3, 4] have not previously been applied to CL with backbone pre-training. QTCL v2 is the first framework to explicitly combine quantum kernels, classical backbone transfer, rehearsal, and QEWC in a unified CL pipeline.

8 Conclusion

We presented QTCL v2, a quantum continual learning framework augmented with a frozen classical backbone pre-trained on the first task. The backbone acts as a domain-invariant feature extractor that consistently improves quantum kernel performance across CL strategies by 10–21% in Average Accuracy and ∼0.15 in Forward Transfer. QTCL + Backbone achieves AA = 0.667 with positive FWT = +0.011, the best-balanced quantum result. The code natively supports IBM Quantum Runtime for real-hardware execution. Future work will evaluate QTCL on real quantum hardware, explore backbone architectures

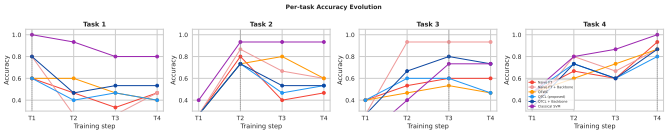

Figure 7: Per-task accuracy evolution over training steps.

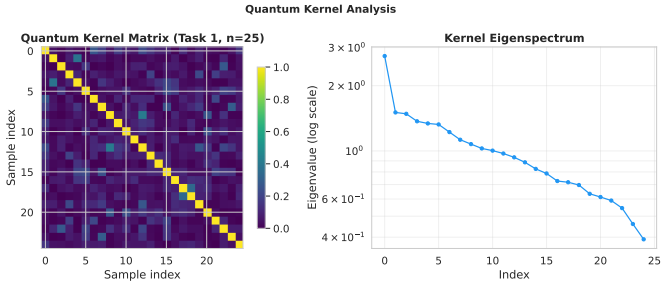

Figure 8: Quantum kernel matrix (left) and eigenspectrum (right) for task 1. Distinct block structure confirms class-discriminative geometry in quantum feature space.

trained on auxiliary quantum-native datasets, and study barren plateau interactions in the CL regime.

References

[1] Michael McCloskey and Neal J Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of learning and motivation*, 24:109–165, 1989.

[2] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. Quantum machine learning. *Nature*, 549(7671):195–202, 2017.

[3] Maria Schuld and Nathan Killoran. Quantum machine learning in feature Hilbert spaces. *Physical Review Letters*, 122(4):040504, 2019.

[4] Vojtěch Havlíček, Antonio D Córcoles, Kristan Temme, Aram W Harrow, Abhinav Kandala, Jerry M Chow, and Jay M Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747): 209–212, 2019.

[5] Andrea Mari, Thomas R Bromley, Josh Izaac, Maria Schuld, and Nathan Killoran. Transfer learning in hybrid classical-quantum neural networks. *Quantum*, 4:340, 2020.

[6] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. *Advances in Neural Information Processing Systems*, 30, 2017.

[7] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in

Python. *Journal of Machine Learning Research*, 12: 2825–2830, 2011.

[8] German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019.

[9] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

A Reproducibility

All experiments use Python 3.12, Qiskit 2.3.1, qiskit-machine-learning 0.9.0, scikit-learn, NumPy seed 42.
Local simulation:

```
pip install qiskit qiskit-machine-learning \
            qiskit-aer matplotlib numpy \
            scipy scikit-learn pandas seaborn
python3 code/qtcl_experiment.py
```

IBM Quantum hardware:

```
pip install qiskit-ibm-runtime
export IBM_QUANTUM=true
export IBM_QUANTUM_TOKEN=<your_token>
export IBM_BACKEND=ibm_brisbane
python3 code/qtcl_experiment.py
```

Code, figures, and data: `https://github.com/danimap27/qtcl-paper`. Estimated compute: ≈ 10 minutes on Intel i5-8500 (single CPU).